Recitation 8 — Interrupts, I2C Sensor Reading & FFT

This recitation covers three things you will need for the course project: **hardware interrupts** for triggering actions on button press, **I2C communication** with the on-board LSM6DSL accelerometer/gyroscope, and **FFT** for extracting frequency information from time-domain sensor data. Everything runs on the B-L475E-IOT01A board using Mbed OS and PlatformIO.

The course project is a gesture-based combination lock. You record a three-gesture sequence by holding the board in a closed fist and performing motions (shake, twist, tilt, etc.). The board saves that sequence as a key. To unlock, you replicate the same three gestures within tolerance. This recitation gives you the building blocks to get there: reading the sensor, detecting when to start/stop, and analyzing motion signals in the frequency domain.

Interrupts

An interrupt is a hardware mechanism that stops what the CPU is currently doing and runs a short function (the ISR — Interrupt Service Routine) in response to an event. In our case, the event is a button press.

The critical rule with ISRs is: **keep them short**. You cannot call `printf`, do I2C reads, or anything that blocks inside an ISR. Complex operations will cause timing issues. The correct pattern is to set a flag in the ISR and check it in your main loop.

Demo 1 — LED Toggle via Button Interrupt

```
#include "mbed.h"

// Create serial and bind it to printf
BufferedSerial serial_port(USBTX, USBRX, 115200);
FileHandle *mbed::mbed_override_console(int) {
    return &serial_port;
}

InterruptIn button(BUTTON1);
DigitalOut led(LED1);

volatile bool buttonPressed = false;

// ISR — just set the flag, nothing else
void button_pressed()
{
    buttonPressed = true;
}

int main()
{
    printf("Interrupt demo started. Press the user button to toggle
LED\r\n");
```

```
    // Attach the handler to the button's falling edge
    button.fall(&button_pressed);

    int counter = 0;

    while (true)
    {
        if (buttonPressed) {
            buttonPressed = false;
            led = !led;
            printf("Button was pressed! LED toggled\r\n");
        }

        counter++;
        printf("Main loop counter: %d\r\n", counter);
        ThisThread::sleep_for(1000ms);
    }
}
```

What is happening here: The `InterruptIn` object monitors the BUTTON1 pin. When the voltage on that pin transitions from high to low (falling edge — because the button connects to ground when pressed), the hardware immediately interrupts whatever the CPU was doing and runs `button_pressed()`. That function sets `buttonPressed = true` and returns. Back in the main loop, we check the flag, toggle the LED, and print. The heavy work (printf, LED control) happens in the main loop, not in the ISR.

Why falling edge? Look at the schematic or user manual (pages 28/57 and 51/57). The button on this board has a pull-up resistor, so the pin sits HIGH when the button is released and goes LOW when pressed. A falling edge detects the press event. You could also use `.rise()` to detect the release.

Why volatile? The variable `buttonPressed` is modified by the ISR and read by the main loop. Without `volatile`, the compiler may cache the value in a register and never see the ISR's update. Always declare shared ISR/main-loop flags as `volatile`.

I2C Sensor Communication

The LSM6DSL is a 6-axis IMU (3-axis accelerometer + 3-axis gyroscope) connected to the STM32 via I2C. To use it you need to:

1. Verify the sensor is connected by reading the `WHO_AM_I` register (should return `0x6A`)
2. Configure the output data rate (ODR) and measurement range by writing to control registers
3. Read data from output registers — each axis is a 16-bit signed value split across two 8-bit registers (little-endian: low byte first, high byte second)

The I2C address is `0x6A` in the datasheet. Mbed uses 8-bit addressing, so you shift left by 1: `0x6A << 1 = 0xD4`.

Key Registers

Register	Address	Purpose

Register	Address	Purpose
`WHO_AM_I`	`0x0F`	Device ID — should return `0x6A`
`CTRL1_XL`	`0x10`	Accelerometer config: ODR and full-scale range
`CTRL2_G`	`0x11`	Gyroscope config: ODR and full-scale range
`OUTX_L_XL` / `OUTX_H_XL`	`0x28` / `0x29`	Accelerometer X-axis (low/high bytes)
`OUTY_L_XL` / `OUTY_H_XL`	`0x2A` / `0x2B`	Accelerometer Y-axis
`OUTZ_L_XL` / `OUTZ_H_XL`	`0x2C` / `0x2D`	Accelerometer Z-axis

Helper Functions

These are the I2C read/write building blocks used throughout this recitation:

```
I2C i2c(PB_11, PB_10);  // I2C2: SDA = PB11, SCL = PB10
#define LSM6DSL_ADDR (0x6A << 1)

// Write a single byte to a register
void write_register(uint8_t reg, uint8_t value) {
    char data[2] = {(char)reg, (char)value};
    i2c.write(LSM6DSL_ADDR, data, 2);
}

// Read a single byte from a register
uint8_t read_register(uint8_t reg) {
    char data = reg;
    i2c.write(LSM6DSL_ADDR, &data, 1, true); // No stop — repeated start
    i2c.read(LSM6DSL_ADDR, &data, 1);
    return (uint8_t)data;
}

// Read a 16-bit value by combining low and high byte registers
int16_t read_16bit_value(uint8_t low_reg, uint8_t high_reg) {
    char low_byte = read_register(low_reg);
    char high_byte = read_register(high_reg);
    return (high_byte << 8) | low_byte;  // little-endian
}
```

The `true` parameter in `i2c.write(..., true)` means "do not send a STOP condition." This keeps the bus held so the subsequent `i2c.read()` happens as a repeated start — which is how the LSM6DSL expects register reads to work.

Demo 2 — Button-Triggered Accelerometer Reading

This combines interrupts and I2C. Press the button to start reading accelerometer data. Press again to stop.

```
#include "mbed.h"

BufferedSerial serial_port(USBTX, USBRX, 115200);
FileHandle *mbed::mbed_override_console(int) {
    return &serial_port;
}

I2C i2c(PB_11, PB_10);

#define LSM6DSL_ADDR (0x6A << 1)
#define WHO_AM_I    0x0F
#define CTRL1_XL    0x10
#define OUTX_L_XL   0x28
#define OUTX_H_XL   0x29
#define OUTY_L_XL   0x2A
#define OUTY_H_XL   0x2B
#define OUTZ_L_XL   0x2C
#define OUTZ_H_XL   0x2D

void write_register(uint8_t reg, uint8_t value) {
    char data[2] = {(char)reg, (char)value};
    i2c.write(LSM6DSL_ADDR, data, 2);
}

uint8_t read_register(uint8_t reg) {
    char data = reg;
    i2c.write(LSM6DSL_ADDR, &data, 1, true);
    i2c.read(LSM6DSL_ADDR, &data, 1);
    return (uint8_t)data;
}

int16_t read_16bit_value(uint8_t low_reg, uint8_t high_reg) {
    char low_byte = read_register(low_reg);
    char high_byte = read_register(high_reg);
    return (high_byte << 8) | low_byte;
}

InterruptIn button(BUTTON1);
DigitalOut led(LED1);

volatile bool isReading = false;
volatile bool buttonPressed = false;

void button_pressed()
{
    buttonPressed = true;
}

int main()
{
    i2c.frequency(400000);

    uint8_t id = read_register(WHO_AM_I);
```

```
    printf("WHO_AM_I = 0x%02X (Expected: 0x6A)\r\n", id);

    if (id != 0x6A) {
        printf("Error: LSM6DSL sensor not found!\r\n");
        while (1) { }
    }

    // Configure accelerometer: 104 Hz, ±2g range
    write_register(CTRL1_XL, 0x40);  // 0100 0000: ODR=104Hz, FS=±2g
    printf("Accelerometer configured: 104 Hz, ±2g range\r\n");

    printf("Press button to start/stop reading\r\n");
    button.fall(&button_pressed);

    int counter = 0;

    while (true)
    {
        if (buttonPressed) {
            buttonPressed = false;
            isReading = !isReading;
            led = isReading;
            printf("%s data reading\r\n", isReading ? "Started" :
"Stopped");
        }

        if (isReading) {
            int16_t acc_x_raw = read_16bit_value(OUTX_L_XL, OUTX_H_XL);
            int16_t acc_y_raw = read_16bit_value(OUTY_L_XL, OUTY_H_XL);
            int16_t acc_z_raw = read_16bit_value(OUTZ_L_XL, OUTZ_H_XL);
            printf("Raw Accel: X=%d, Y=%d, Z=%d\r\n", acc_x_raw, acc_y_raw,
acc_z_raw);
        }
        else {
            counter++;
            printf("Main loop doing counter: %d\r\n", counter);
        }

        ThisThread::sleep_for(1000ms);
    }
}
```

What is happening here: This is the same interrupt pattern from Demo 1, but now the flag toggles a reading state. When `isReading` is true, the main loop reads all three accelerometer axes every second and prints the raw 16-bit values. The LED mirrors the reading state so you get visual feedback. When you press the button again, reading stops and the loop goes back to printing a counter.

Why this matters for the project: Your gesture lock needs a way to invoke "Record" and "Unlock" modes. This demo shows exactly that pattern — a button toggles between idle and active sensor reading. For the project you would extend this to have two modes (record vs unlock) instead of just on/off.

FFT — Fast Fourier Transform

What FFT Does

When you read accelerometer data over time, you get a **time-domain signal** — a sequence of values that tells you what the acceleration was at each moment. This is useful but it is hard to directly compare two gestures in the time domain because even small differences in timing, speed, or starting position make the raw signals look very different.

FFT converts a time-domain signal into the **frequency domain**. Instead of "what was the acceleration at time t?" you get "how much energy is at frequency f?" A sharp shake might have most of its energy around 5-10 Hz. A slow tilt might be below 2 Hz. A rapid vibration might peak at 20 Hz. These frequency signatures are much more stable and repeatable than the raw time waveforms, which is why FFT is useful for gesture recognition.

The Math (Simplified)

The FFT takes N time-domain samples and produces N/2 frequency bins. Each bin represents a specific frequency:

- **Frequency resolution** = Sample Rate / FFT Size
- **Bin k represents** frequency `k × (Sample Rate / FFT Size)` Hz
- **Bin 0** is the DC component (average value — usually not interesting for gestures)
- **Bin N/2** is the Nyquist frequency (half the sample rate)

For example with a 104 Hz sample rate and 256-point FFT: each bin is 104/256 ≈ 0.406 Hz wide, and you can detect frequencies up to 52 Hz.

CMSIS-DSP Setup

The STM32L475 has an ARM Cortex-M4F processor with hardware floating-point support. ARM provides the CMSIS-DSP library which includes optimized FFT routines that take advantage of this hardware.

To add CMSIS-DSP to your PlatformIO project:

1. Download `CMSIS-DSP-main` from Brightspace (or clone from GitHub)
2. Place the folder in your project's `lib/` directory
3. Add to `platformio.ini`:

```
build_flags =
    -DARM_MATH_CM4
    -Ilib/CMSIS-DSP-main/Include
    -Ilib/CMSIS-DSP-main/Source
```

`-DARM_MATH_CM4` tells the library that our board has a Cortex-M4 processor so it can use the right optimizations.

4. Include `"arm_math.h"` in your source files. There are other headers in the library for specific operations (FIR filters, etc.) — see the CMSIS-DSP documentation for details.

Demo 3 — FFT on a Single Sine Wave

This is the simplest FFT example. We generate a known 1 kHz sine wave in software, run FFT on it, and verify that the output correctly identifies 1 kHz as the dominant frequency.

```
#include "mbed.h"
#include "arm_math.h"

BufferedSerial serial_port(USBTX, USBRX, 115200);
FileHandle *mbed::mbed_override_console(int) { return &serial_port; }

#define FFT_SIZE 256
#define SAMPLE_RATE 10000

float32_t input_fft[FFT_SIZE];
float32_t fft_out[FFT_SIZE];
float32_t magnitude[FFT_SIZE / 2];

float32_t freq = 1000.0f;  // 1 kHz test signal

arm_rfft_fast_instance_f32 FFT_Instance;

void make_sine_wave(float freq) {
    for (int i = 0; i < FFT_SIZE; i++) {
        float t = (float)i / SAMPLE_RATE;
        input_fft[i] = arm_sin_f32(2 * PI * freq * t);
    }
}

void run_fft() {
    arm_rfft_fast_f32(&FFT_Instance, input_fft, fft_out, 0);
    arm_cmplx_mag_f32(fft_out, magnitude, FFT_SIZE / 2);
}

void show_results() {
    float resolution = SAMPLE_RATE / FFT_SIZE;

    printf("Bin\tFreq (Hz)\tMagnitude\n");
    for (int i = 0; i < 28; i++) {
        printf(" %d\t%.2f\t\t%.4f\n", i, i * resolution, magnitude[i]);
    }

    uint32_t max_index = 0;
    float32_t max_val = 0.0f;
    arm_max_f32(magnitude, FFT_SIZE / 2, &max_val, &max_index);

    printf("Peak Frequency: %.2f Hz\n\n", max_index * resolution);
    printf("Max magnitude: %.1f at bin %lu (%.2f Hz)\r\n",
        max_val, max_index, max_index * resolution);
}

int main() {
```

```
    arm_rfft_fast_init_f32(&FFT_Instance, FFT_SIZE);

    make_sine_wave(freq);
    run_fft();
    show_results();

    while (true) {
        ThisThread::sleep_for(1000ms);
    }
}
```

What is happening here:

1. `arm_rfft_fast_init_f32()` initializes the FFT instance with lookup tables for the given size. This must be called once before any FFT computation. The size must be a power of 2.
2. `make_sine_wave()` generates 256 samples of a pure 1 kHz sine wave using `arm_sin_f32()`. At a 10 kHz sample rate each sample is 0.1 ms apart, so 256 samples cover 25.6 ms.
3. `arm_rfft_fast_f32()` computes the real-valued FFT. The input is 256 real samples, the output is 256 values representing complex frequency bins (interleaved real/imaginary pairs). The last argument `0` means forward transform (time → frequency). Use `1` for the inverse.
4. `arm_cmplx_mag_f32()` converts each complex bin (real + imaginary pair) into a single magnitude value. This gives you 128 magnitude values — one per frequency bin.
5. `arm_max_f32()` finds the bin with the highest magnitude. At 10 kHz / 256 = 39.06 Hz per bin, a 1 kHz signal lands in bin 25 or 26 (1000 / 39.06 ≈ 25.6). The output should confirm this.

Demo 4 — FFT on a Composite Wave

What happens when a signal contains multiple frequencies at once? This is closer to real sensor data, where a gesture might produce vibrations at several frequencies simultaneously.

```
#include "mbed.h"
#include "arm_math.h"

BufferedSerial serial_port(USBTX, USBRX, 115200);
FileHandle *mbed::mbed_override_console(int) { return &serial_port; }

#define FFT_SIZE     256
#define SAMPLE_RATE  10000.0f

float32_t input_fft[FFT_SIZE];
float32_t fft_out[FFT_SIZE];
float32_t magnitude[FFT_SIZE / 2];

arm_rfft_fast_instance_f32 FFT_Instance;

void make_composite_wave() {
    float f1 = 500.0f;
```

```
    float f2 = 1000.0f;
    float f3 = 1500.0f;
    float f4 = 2500.0f;

    for (int i = 0; i < FFT_SIZE; i++) {
        float t = (float)i / SAMPLE_RATE;
        input_fft[i] =
              0.7f * arm_sin_f32(2 * PI * f1 * t)
            + 1.0f * arm_sin_f32(2 * PI * f2 * t)
            + 0.5f * arm_sin_f32(2 * PI * f3 * t)
            + 0.3f * arm_sin_f32(2 * PI * f4 * t);
    }
}

void plot_composite_wave() {
    for (int i = 0; i < FFT_SIZE; i++) {
        printf(">signal:%.4f\n", input_fft[i]);
        ThisThread::sleep_for(1ms);
    }
}

void run_fft() {
    arm_rfft_fast_f32(&FFT_Instance, input_fft, fft_out, 0);
    arm_cmplx_mag_f32(fft_out, magnitude, FFT_SIZE / 2);
}

void plot_peak_frequencies() {
    float resolution = SAMPLE_RATE / FFT_SIZE;

    for (int i = 1; i < FFT_SIZE / 2 - 1; i++) {
        if (magnitude[i] > magnitude[i - 1] &&
            magnitude[i] > magnitude[i + 1] &&
            magnitude[i] > 0.2f) {
            float freq = i * resolution;
            printf(">peak_freq:%.1f 1.0\n", freq);
            ThisThread::sleep_for(10ms);
        }
    }
}

int main() {
    arm_rfft_fast_init_f32(&FFT_Instance, FFT_SIZE);

    make_composite_wave();
    plot_composite_wave();

    run_fft();
    plot_peak_frequencies();

    while (true) {
        ThisThread::sleep_for(1s);
    }
}
```

What is happening here: We create a signal that is the sum of four sine waves at 500, 1000, 1500, and 2500 Hz with different amplitudes (0.7, 1.0, 0.5, 0.3). In the time domain this looks like a messy, complicated waveform. After FFT, the magnitude spectrum shows four clear peaks at exactly those frequencies. The FFT decomposes the messy signal back into its component frequencies.

The `plot_peak_frequencies()` function uses a simple peak detection algorithm: a bin is a peak if it is larger than both its neighbors and above a minimum threshold (0.2). This is the same idea you would use to find dominant gesture frequencies from real sensor data.

The `>signal:` and `>peak_freq:` prefixes are Teleplot format — it will automatically plot these as separate channels.

Demo 5 — FFT on a Real-Valued Dataset

Demos 3 and 4 used synthetic sine waves. This demo runs FFT on an actual dataset of 1024 samples — similar to what you will do in the project when you collect accelerometer data, store it in an array, and run FFT on it.

```
#include "mbed.h"
#include "arm_math.h"

BufferedSerial serial_port(USBTX, USBRX, 115200);
FileHandle *mbed::mbed_override_console(int) { return &serial_port; }

#define TEST_LENGTH_SAMPLES 1024
#define FFT_SIZE 1024

// In your project, replace this with collected accelerometer data
float32_t testInput_f32_10khz[TEST_LENGTH_SAMPLES] = {
    -0.865129623056441, -2.655020678073846, 0.600664612949661, ...
    // (1024 real-valued samples)
};

float32_t fft_out[FFT_SIZE];
float32_t magnitude[FFT_SIZE / 2];

float32_t SAMPLE_RATE = 48000.0f;

arm_rfft_fast_instance_f32 FFT_Instance;

void run_fft() {
    arm_rfft_fast_f32(&FFT_Instance, testInput_f32_10khz, fft_out, 0);
    arm_cmplx_mag_f32(fft_out, magnitude, FFT_SIZE / 2);
}

void show_results() {
    float32_t resolution = SAMPLE_RATE / FFT_SIZE;
    float32_t maxValue;
    uint32_t maxIndex;

    arm_max_f32(magnitude, FFT_SIZE / 2, &maxValue, &maxIndex);
```

```
    printf("Max magnitude: %.1f at bin %lu (%.2f Hz)\r\n",
           maxValue, maxIndex, maxIndex * resolution);

    printf("Bin\tFreq (Hz)\tMagnitude\n");
    for (int i = 0; i < 28; i++) {
        printf(" %d\t%.2f\t\t%.4f\n", i, i * resolution, magnitude[i]);
    }
}

int main(void) {
    arm_status status = arm_rfft_fast_init_f32(&FFT_Instance, FFT_SIZE);

    if (status != ARM_MATH_SUCCESS) {
        printf("FFT initialization failed\r\n");
        while(1);
    }

    run_fft();
    show_results();

    while (true) {
        ThisThread::sleep_for(1000ms);
    }
}
```

Why this matters for the project: This is the pattern you will follow. During the "Record Key" phase, you sample the accelerometer into an array. Then you run FFT on that array to extract frequency features. You save those features as the key. During the "Unlock" phase, you sample again, run FFT again, and compare the frequency features to the saved key.

Note the error checking on `arm_rfft_fast_init_f32()` — it returns `ARM_MATH_SUCCESS` if the size is valid (must be a power of 2: 128, 256, 512, 1024, etc.) and fails otherwise. Always check this.

Putting It All Together — The Main Demo

The `main.cpp` in this repo combines everything into a live, running demo: it reads the accelerometer over I2C at 104 Hz, applies a moving average filter to smooth the signal, and runs a 256-point FFT to find the dominant vibration frequency. Here is what each piece does:

Data Acquisition

```
// Read raw Z-axis acceleration
int16_t raw_val = read_int16(OUTZ_L_XL);

// Convert to g (±8g range, sensitivity = 0.244 mg/LSB)
float acc_z = raw_val * 0.244f / 1000.0f;
```

The raw 16-bit value from the sensor is in LSB (Least Significant Bits). To convert to physical units (g), you multiply by the sensitivity factor from the datasheet. For ±8g range, that is 0.244 mg/LSB.

Pre-Processing — Remove DC Offset

```
float acc_z_centered = acc_z - 1.0f;
```

When the board is sitting still, the Z-axis reads approximately 1g (gravity). FFT works best on signals centered around zero. Subtracting 1.0 removes the DC offset so the FFT focuses on the actual motion, not the constant gravity component. Without this, bin 0 (DC) would dominate the spectrum and hide the interesting frequencies.

Moving Average Filter (Time Domain)

```
#define MA_WINDOW 10

float ma_buffer[MA_WINDOW] = {0};
int ma_idx = 0;
float ma_sum = 0.0f;

// In the loop:
ma_sum -= ma_buffer[ma_idx];       // subtract oldest sample
ma_buffer[ma_idx] = acc_z;         // store newest sample
ma_sum += ma_buffer[ma_idx];       // add newest to sum
ma_idx = (ma_idx + 1) % MA_WINDOW; // advance circular index

float filtered_acc_z = ma_sum / MA_WINDOW;
```

This is a circular buffer implementing a moving average. At each step you subtract the oldest value, add the newest, and divide by the window size. This is O(1) per sample regardless of window size — you do not need to sum the entire buffer each time. The modulo operator wraps the index back to 0 when it reaches the end, so the buffer reuses memory continuously.

A 10-sample window at 104 Hz means each output value is the average of the last ~96 ms of data. This smooths out high-frequency noise while preserving the overall shape of the signal. Useful for visualizing trends, but for gesture features you will likely want FFT instead.

FFT (Frequency Domain)

```
#define FFT_SIZE 256

float fft_input[FFT_SIZE];
int fft_idx = 0;

// In the loop — accumulate samples:
fft_input[fft_idx] = acc_z_centered;
```

```
    fft_idx++;

    if (fft_idx >= FFT_SIZE) {
        // 1. Compute FFT
        arm_rfft_fast_f32(&S, fft_input, fft_output, 0);

        // 2. Compute magnitudes
        arm_cmplx_mag_f32(fft_output, fft_mag, FFT_SIZE / 2);

        // 3. Find dominant frequency (skip bin 0 = DC)
        float max_val = 0.0f;
        int max_bin = 0;
        for (int i = 1; i < FFT_SIZE / 2; i++) {
            if (fft_mag[i] > max_val) {
                max_val = fft_mag[i];
                max_bin = i;
            }
        }

        // 4. Convert to Hz (only if above noise floor)
        if (max_val > 1.0f) {
            current_dominant_freq = max_bin * FREQ_BIN_SIZE;
        } else {
            current_dominant_freq = 0.0f;
        }

        fft_idx = 0;  // reset for next window
    }
```

What is happening here: Samples arrive one at a time from the sensor. We accumulate them into `fft_input[]` until we have 256. Then we run the full FFT pipeline: time → frequency → magnitude → peak detection. The noise threshold (`max_val > 1.0f`) prevents reporting spurious frequencies when the board is sitting still and there is no real motion. After processing, we reset `fft_idx` to start filling the buffer again. This means we get one frequency update every 256/104 ≈ 2.46 seconds.

DSP Parameters Summary

Parameter	Value	How to Calculate
Sample Rate (ODR)	104 Hz	Set via `CTRL1_XL` register
Accelerometer Range	±8 g	Sensitivity: 0.244 mg/LSB
Moving Average Window	10 samples	~96 ms smoothing at 104 Hz
FFT Size	256 points	Must be power of 2
Frequency Resolution	~0.406 Hz/bin	Sample Rate / FFT Size
FFT Update Period	~2.46 s	FFT Size / Sample Rate
Max Detectable Frequency	52 Hz	Sample Rate / 2 (Nyquist)

Visualization with Teleplot

The `main.cpp` outputs data in Teleplot format:

```
printf(">Raw_Acc:%.2f\n", acc_z);
printf(">Filtered_Acc:%.2f\n", filtered_acc_z);
printf(">Freq_Hz:%.2f\n", current_dominant_freq);
```

The `>channel_name:value` format is automatically recognized by the Teleplot VS Code extension. It will create live updating graphs for each channel. `Raw_Acc` and `Filtered_Acc` update every ~10 ms. `Freq_Hz` updates every ~2.5 seconds (once per FFT window).

Build & Run

Prerequisites

- PlatformIO (CLI or VS Code extension)
- ST-Link drivers (for flashing/debugging)
- Teleplot VS Code extension (optional, for live graphs)

Commands

```
# Build the project
pio run -e disco_l475vg_iot01a

# Upload to the board
pio run -e disco_l475vg_iot01a --target upload

# Open serial monitor
pio device monitor -b 115200
```

Enabling Floating-Point Printf

Add this `mbed_app.json` to your project root:

```
{
    "target_overrides": {
        "*": {
            "platform.minimal-printf-enable-floating-point": true
        }
    }
}
```

Without this, `printf` with `%f` will print `0.000000` for everything on Mbed OS.

How This Connects to the Project

Each piece of this recitation maps directly to a part of the gesture lock project:

Recitation Concept	Project Application
Button interrupt (Demo 1)	Triggering "Record" and "Unlock" modes
Toggle state on press (Demo 2)	Switching between idle, recording, and unlocking
I2C accelerometer reads (Demo 2)	Capturing gesture data from the IMU
FFT on known signals (Demos 3–4)	Understanding how gestures appear in frequency domain
FFT on stored data (Demo 5)	Analyzing recorded gesture sequences
Live FFT pipeline (main.cpp)	Real-time gesture detection during unlock
Moving average filter	Smoothing sensor noise before feature extraction
Teleplot visualization	Debugging and tuning your gesture recognition

References

- CMSIS-DSP GitHub Repository
- CMSIS-DSP API Documentation
- LSM6DSL Datasheet
- B-L475E-IOT01A User Manual
- PlatformIO Mbed Documentation